Introduction

The Virtual Input/Output (VIO) core is a customizable core that can both monitor and drive internal FPGA signals in real time. The number and width of the input and output ports are customizable in size to interface with the FPGA design. Because the VIO core is synchronous to the design being monitored and/or driven, all design clock constraints that are applied to your design are also applied to the components inside the VIO core. Run time interaction with this core requires the use of the Vivado® logic analyzer feature. The following figure is a block diagram of the new VIO core.

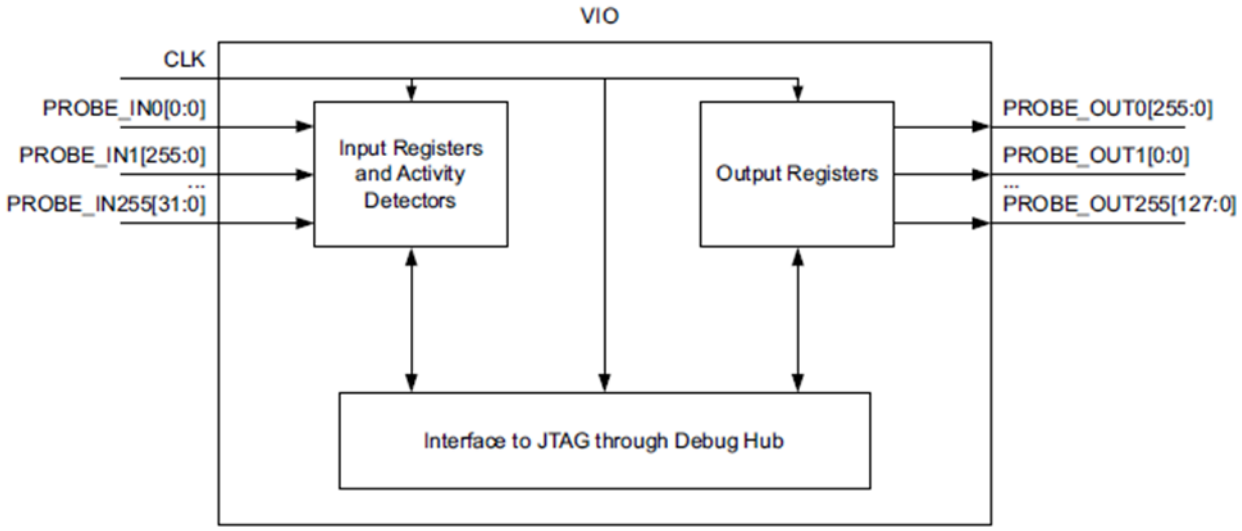


Figure 21: VIO Block Diagram

This lab walks you through the steps of instantiating and configuring the VIO core. It walks you through the steps of connecting the I/Os of the design to the VIO core. This way, you can debug your design when you do not have access to the hardware or the hardware is remotely located.

The following ports are created:

- One 4-bit PROBE_IN0 port. This has two bits to monitor the 2-bit Sine Wave selector outputs from the finite state machine (FSM) and other two bits to mimic the state of the other two LEDs on the board. We will configure these 4-bit signals as LEDs during run time to mimic the LEDs displayed on the KC705 board.

- One 2-bit PROBE_OUT0 port to drive the input buttons on the FSM. We will configure it so one bit can be used as a toggle switch during run time to mimic the "PUSH_BUTTON", SW3, and second bit will be used as the "PUSH_BUTTON", SW6.

Step 1: Creating a Project with the Vivado New Project Wizard

To create a project, use the New Project wizard to name the project, to add RTL source files and constraints, and to specify the target device.

1. Invoke Vivado IDE.
2. In the **Quick Start** tab, click **Create Project** to start the New Project wizard. Click **Next**.
3. In the **Project Name** page, name the new project **proj_hdl_vio** and provide the project location (`C:/Vivado_Debug`). Ensure that **Create project subdirectory** is selected. Click **Next**.
4. In the **Project Type** page, specify the **Type of Project** to create as **RTL Project**. Click **Next**.
5. In the **Add Sources** page:
   a. Set **Target Language** to **VHDL**.
   b. Click **Add Files**.
   c. In the **Add Source** Files dialog box, navigate to the `/src/lab3 directory`.
   d. Select all VHD source files, and click **OK**.
   e. Verify that the files are added, and **Copy Sources into Project** is selected. Click **Next**.
6. Click the green "+" sign, and then click **Add Files**.
7. In the **Add Source Directories** dialog box, navigate to the `/src/lab3` directory and choose the `sine_high`, `sine_low`, `sine_mid,` and `ila_0` directories. Click **Select**.
8. Verify that the files are added and **Copy sources into project** is selected. Click **Next**.
9. In the **Add Constraints** dialog box, click the "**+**" sign, and then click **Add Files**.
10. Navigate to `/src/lab3` directory and select `sinegen_demo_kc705.xdc`. Click **Next**.
11. In the **Default Part** page, specify the **xc7k325tffg900-2** part for the KC705 platform. You can also select **Boards** and then select **Kintex-7 KC705 Evaluation Platform**. Click **Next**.
12. Review the **New Project Summary** page. Verify that the data appears as expected, per the steps above. Click **Finish**.

    ***Note**: It might take a moment for the project to initialize.*

13. In the **Sources** window in Vivado IDE, expand sinegen_demo_inst_vio to see the source files for this lab. Note that `ila_0` core has been added to the project. However, vio_0 (the VIO core) is missing.

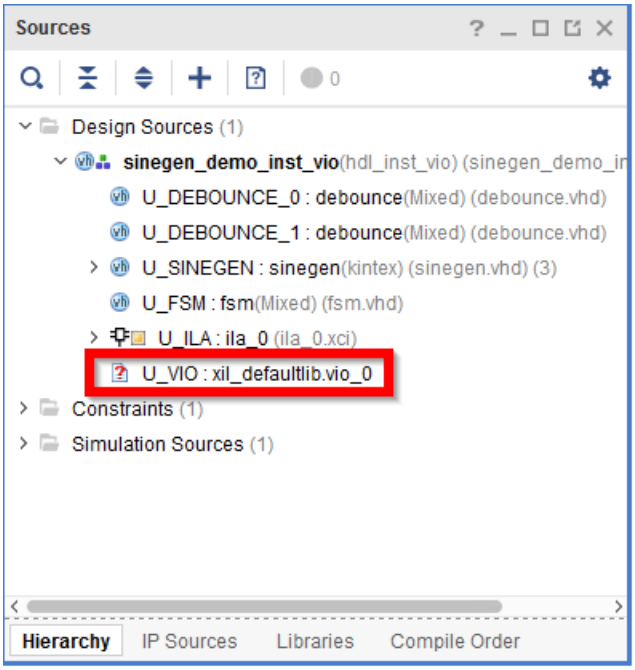


Figure 22: Missing Source for VIO Core

14. In this step, you will instantiate and configure this VIO core. From the **Flow Navigator**, click **IP Catalog**, expand **Debug & Verification**, then expand **Debug**, and double-click **VIO**. The **Customize IP** dialog box opens.
15. On the **General Options** tab, leave the **Component Name** to its default value of **vio_0**, **set Input Probe Count** to **1**, **Output Probe Count** to **1**, and select the **Enable Input Probe Activity Detectors** check box.

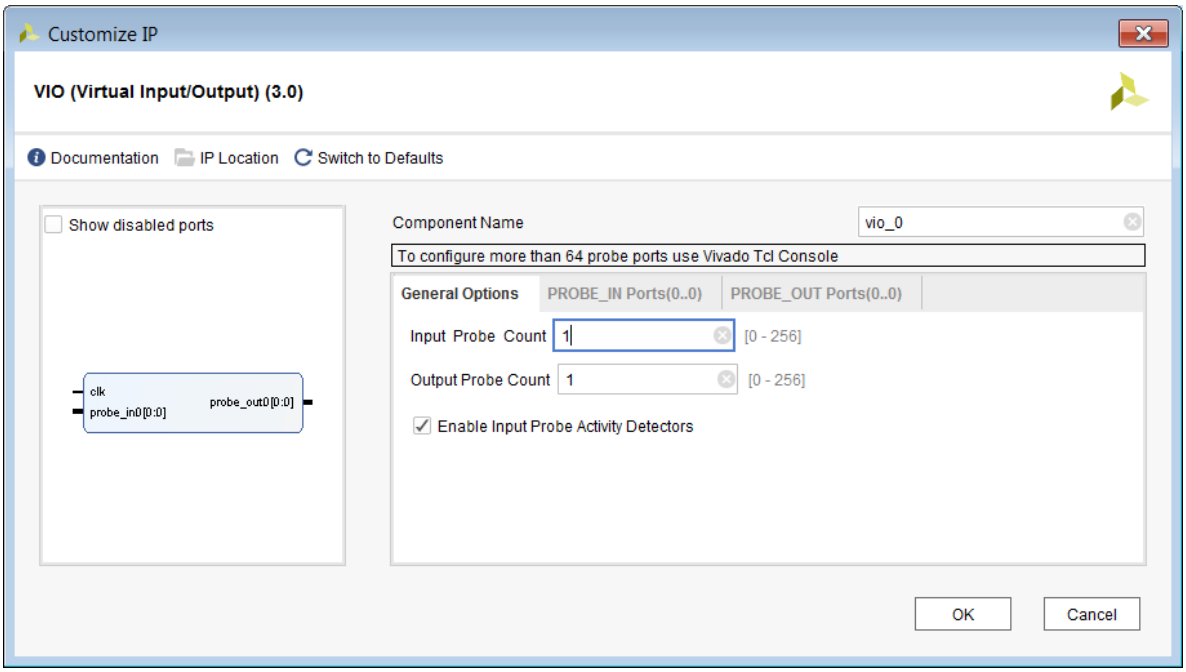


Figure 23: Configure General Options of the VIO Core

16. On the **PROBE_IN Ports** tab, set **Probe Width** to **4** bits wide.

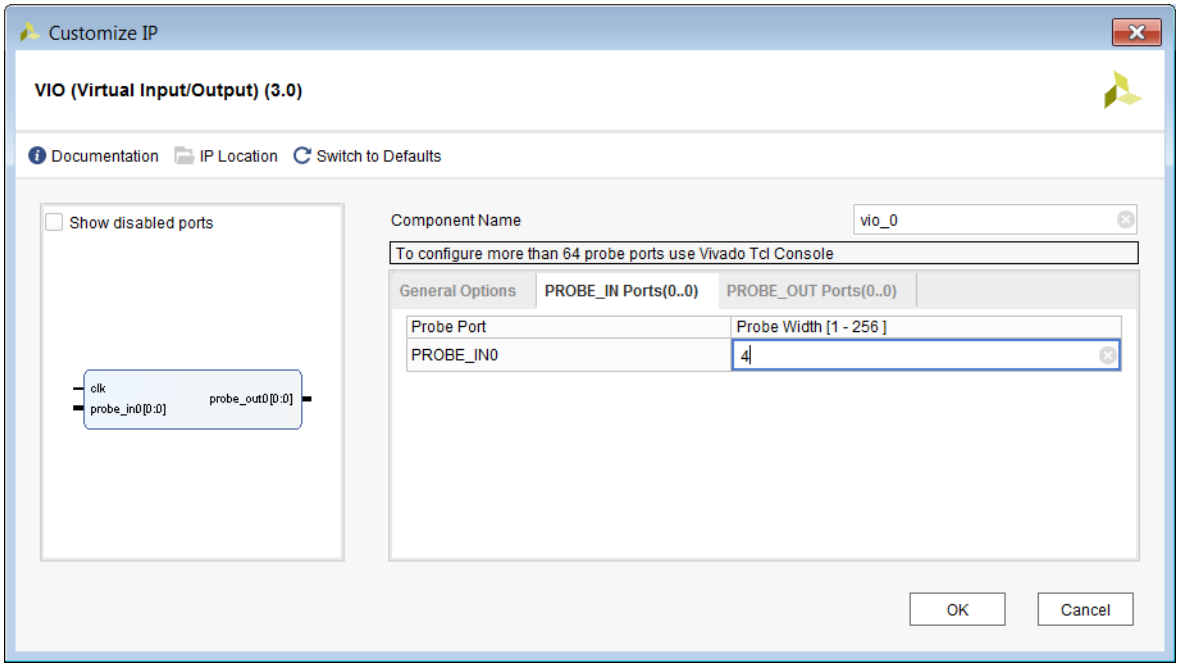


Figure 24: Configure PROBE_IN Ports of the VIO Core

17. On **the PROBE _OUT Ports**, set **Probe Width** to **2** bits wide with an initial value of **0** in hex format.

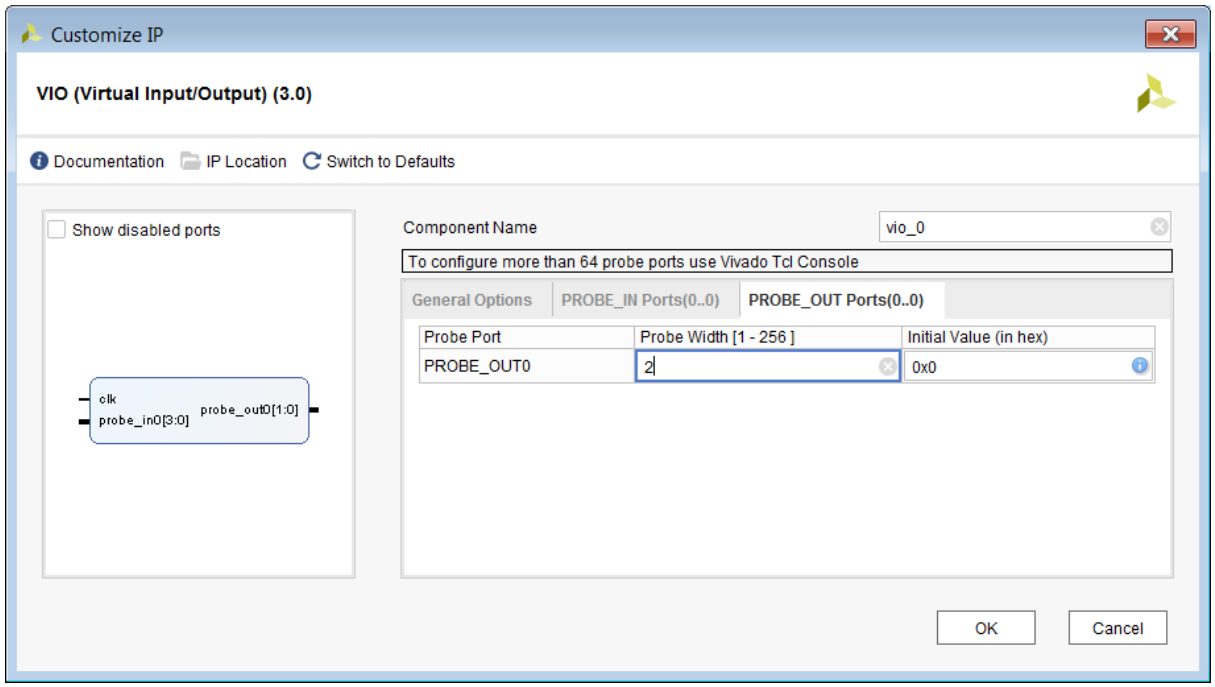


Figure 25: Configure the PROBE_OUT Ports of the VIO Core

18. Click **OK** to generate the IP. The **Generate Output Products** dialog box will appear. Click **Generate**.

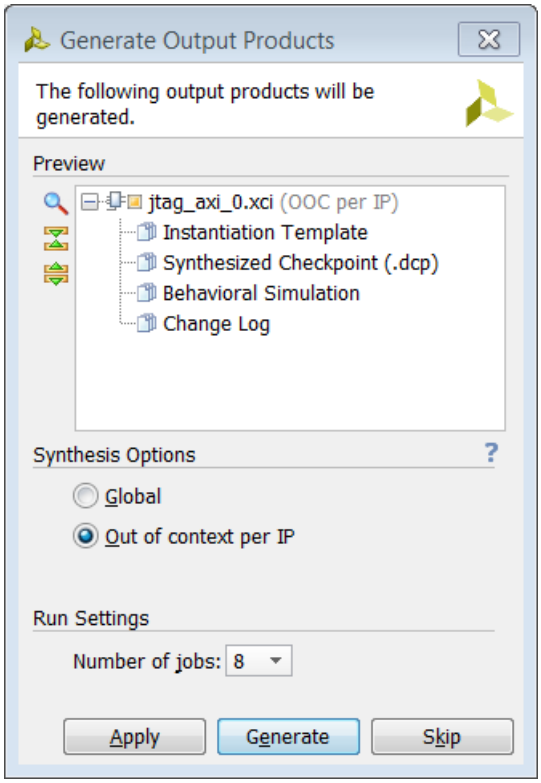


Figure 26: Generate Output Products for the VIO Core

Output product generation should take less than a minute. At this point, you have finished customizing the VIO. This core has already been instantiated in the top level design as shown in the following figure.

```
-----------------------------------------------------------
-- VIO
-----------------------------------------------------------
U_VIO : vio_0
  port map
  (
    CLK => clk,
    PROBE_IN0(3) => DONT_EAT,
    PROBE_IN0(2) => GPIO_BUTTONS_re(1),
    PROBE_IN0(1 downto 0) => sineSel,
    PROBE_OUT0(1) => push_button_reset,
    PROBE_OUT0(0) => push_button_vio
  );
```

Figure 27: VIO Instantiation in the Top Level Design

At this point, the **Sources** window should look as shown in the following figure.

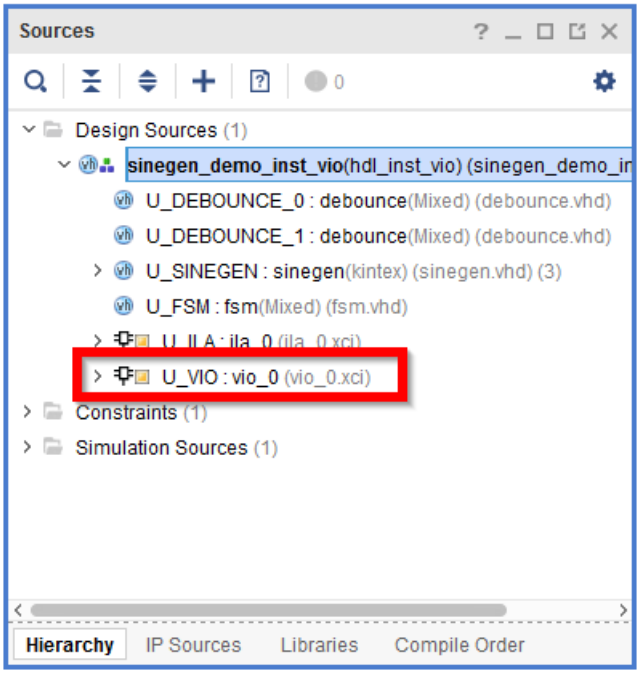


Figure 28: Instantiated VIO Core in the Sources Window

19. Double-click **sinegen_demo_inst.vhd** in the **Sources** window, to open it and inspect the instantiation and port mapping of the ILA core in the HDL code.

```
------------------------------------------------------------
-- ILA
------------------------------------------------------------
U_ILA : ila_0
  port map
  (
    CLK => clk,
    PROBE0 => sineSel,
    PROBE1 => sine,
    PROBE2 => GPIO_BUTTONS_db,
    PROBE3 => GPIO_BUTTONS_re,
    PROBE4 => GPIO_BUTTONS_dly,
    PROBE5 => GPIO_BUTTONS
  );
```

Figure 29: Hook signals that need to be debugged in the ILA

Step 2: Synthesize, Implement, and Generate Bitstream

1. From the **Program and Debug** drop-down list, in **Flow Navigator**, click **Generate Bitstream**. This synthesizes, implements, and generates a bitstream for the design.
2. The **Missing Implementation Results** dialog box appears. Click **OK**.
3. After bitstream generation completes, the **Bitstream Generation Completed** dialog box appears. **Open Implemented Design** is selected by default. Click **OK**.
4. Inspect the Timing Summary report and make sure that all timing constraints have been met.

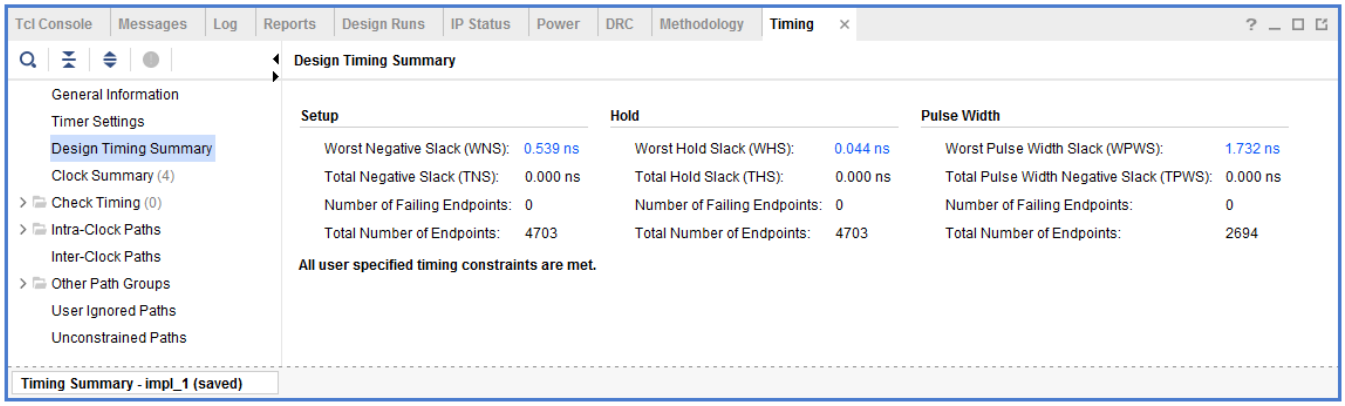


Figure 30: Report Timing Summary Dialog Box

5. Proceed to Lab 5: Using Vivado Logic Analyzer to Debug Hardware chapter to complete the rest of the steps for debugging the design. Skip forward to **Verifying the VIO Core Activity (Only applicable to Lab 3)** section to complete the rest of this lab.